

Apuntes DBA

Bases de Datos para Entrevistas y Uso Profesional

Aldo Zetina Muciño

zetinaa3@gmail.com

16 de mayo de 2026

Índice general

1. Introducción a Bases de Datos	4
1.1. ¿Qué es una base de datos?	4
1.2. Tipos de bases de datos	5
1.3. DBMS vs archivo plano	5
2. Modelo Relacional	8
2.1. Conceptos fundamentales	8
2.2. Llaves	8
2.3. Integridad referencial	9
3. SQL Fundamentos	12
3.1. Sublenguajes SQL	12
3.2. DDL — definir estructura	12
3.3. DML + DQL — manipular y consultar	13
4. SQL Avanzado	16
4.1. CTEs (Common Table Expressions)	16
4.2. Window Functions	16
4.3. EXPLAIN ANALYZE	17
5. Modelado UML y Diagramas ER	20
5.1. Diagrama Entidad-Relación (ER)	20
5.2. Notaciones	20
5.3. Cardinalidad en Crow's Foot	21
6. Normalización	23
6.1. ¿Por qué normalizar?	23
6.2. Formas normales	23
6.3. Dependencias funcionales	24
7. Índices	27
7.1. ¿Qué es un índice?	27
7.2. Tipos de índices en PostgreSQL	27
7.3. Cuándo crear (y no crear) un índice	28
8. Transacciones y Control de Concurrency	31
8.1. ACID	31
8.2. Niveles de aislamiento	32
8.3. Locks y Deadlocks	32
9. PostgreSQL — Características Avanzadas	35
9.1. Por qué PostgreSQL	35
9.2. JSONB	35

9.3. VACUUM y mantenimiento	36
10. MySQL / MariaDB	38
10.1. InnoDB vs MyISAM	38
10.2. Diferencias clave con PostgreSQL	38
10.3. Particionamiento	39
11. Bases de Datos NoSQL	41
11.1. ¿Cuándo NoSQL?	41
11.2. Tipos y cuándo usar cada uno	41
11.3. Teorema CAP	42
12. Diseño de Base de Datos — Caso Práctico	45
12.1. Proceso de diseño	45
12.2. Caso: Sistema de Biblioteca	45
13. Administración de Bases de Datos	50
13.1. Backups	50
13.2. Usuarios, roles y permisos	51
13.3. Monitoreo básico	51
14. Preguntas de Entrevista DBA	54
14.1.  SQL	54
14.2.  Índices y rendimiento	54
14.3.  Transacciones y concurrencia	55
14.4.  Modelado y diseño	55
14.5.  NoSQL y arquitectura distribuida	55
14.6.  Administración y operaciones	55

Capítulo 1

Introducción a Bases de Datos

1.1. ¿Qué es una base de datos?

Una base de datos es un sistema organizado para almacenar, gestionar y recuperar información de forma persistente y estructurada. La clave es la palabra *organizado*: los datos no solo se guardan, sino que se almacenan con una estructura que permite recuperarlos eficientemente, garantiza su integridad y permite consultarlos de formas arbitrarias.

El **DBMS** (Database Management System) es el software que se interpone entre la aplicación y el almacenamiento físico. La aplicación nunca toca directamente los archivos en disco; todo pasa por el DBMS, que administra acceso concurrente, integridad de datos, recuperación ante fallos y seguridad. Esta separación de responsabilidades es fundamental: permite cambiar el motor de base de datos sin modificar la lógica de negocio de la aplicación.

Componentes internos de un DBMS

- **Motor de almacenamiento:** decide cómo se organizan los datos en disco (páginas, extents, tablespaces). InnoDB y MyISAM son motores distintos en MySQL.
- **Gestor de consultas (Query Planner):** analiza el SQL, genera un árbol de operaciones y elige el plan de ejecución más barato según estadísticas.
- **Gestor de transacciones:** coordina el acceso concurrente garantizando ACID. Usa locks o MVCC para aislar transacciones.
- **Gestor de buffer (Buffer Pool):** mantiene páginas de disco en memoria RAM para evitar lecturas físicas costosas. Es el componente más crítico para el rendimiento.
- **Gestor de recuperación (WAL):** escribe un log de cambios antes de modificar los datos. Si el servidor se cae, el log permite reconstruir el estado consistente.
- **BD:** colección estructurada de datos relacionados con semántica definida por el esquema
- **RDBMS:** DBMS basado en modelo relacional — PostgreSQL, MySQL, Oracle, SQL Server

1.2. 📄 Tipos de bases de datos

No existe un tipo de base de datos universalmente superior. Cada modelo está optimizado para un patrón de acceso específico y un tipo de dato diferente. Elegir el modelo equivocado puede resultar en un sistema que funciona pero que es imposible de escalar o mantener a mediano plazo. La tendencia moderna es **polyglot persistence**: usar múltiples tipos de BD en el mismo sistema, cada uno para lo que hace mejor.

- **Relacional (SQL)**: tablas con esquema fijo, joins entre tablas, transacciones ACID, lenguaje SQL estándar. Ideal para datos estructurados con relaciones complejas. PostgreSQL, MySQL, Oracle, SQL Server.
- **Documental**: documentos JSON/BSON, esquema flexible por documento, bueno para datos heterogéneos o con estructura variable. MongoDB, CouchDB, Firestore.
- **Clave-valor**: acceso $O(1)$ por clave, sin estructura interna de valor, extremadamente rápido en lectura. Redis, DynamoDB, Memcached.
- **Columnar**: optimizado para leer muchas filas de pocas columnas (analítica), no para transacciones. Cassandra, ClickHouse, Apache HBase.
- **Grafos**: entidades como nodos, relaciones como aristas con propiedades. Ideal cuando las relaciones son tan importantes como los datos. Neo4j, Amazon Neptune.
- **Series de tiempo**: datos con timestamp, optimizado para ingestión masiva y consultas por rango temporal. InfluxDB, TimescaleDB (extensión de PG).

💡 ¿Cuándo elegir SQL vs NoSQL?

Usa SQL (relacional) por defecto: es maduro, tiene garantías ACID, soporte de joins y herramientas consolidadas. Considera NoSQL cuando: el esquema varía mucho entre registros (documental), necesitas caché de baja latencia (clave-valor), escalado horizontal masivo de escritura (columnar), o las relaciones entre entidades son el dato principal (grafos). En sistemas complejos, la respuesta es frecuentemente “ambos”.

1.3. ⚖️ DBMS vs archivo plano

Es válido preguntarse por qué no usar simplemente archivos CSV o JSON en disco. La respuesta es la **conurrencia**: cuando dos procesos leen y escriben un archivo simultáneamente sin coordinación, los datos se corrompen. Un DBMS resuelve este problema y muchos más que no son obvios hasta que ocurren en producción.

Imagine un sistema de inventario con archivos CSV: si dos cajeros descuentan el último artículo al mismo tiempo, ambos leen “stock: 1”, ambos calculan “stock: 0” y ambos escriben. El resultado es stock negativo. Un DBMS con transacciones garantiza que solo uno de los dos vea el artículo disponible.

- **Concurrencia controlada:** múltiples usuarios simultáneos sin corrupción de datos (locks, MVCC)
- **Integridad referencial:** las FK garantizan que no existan huérfanos (pedidos sin cliente)
- **Recuperación ante fallos:** WAL/journaling reconstruye el estado consistente tras una caída
- **Consultas declarativas:** SQL describe *qué* quieres; el motor elige *cómo* obtenerlo
- **Seguridad granular:** permisos a nivel de tabla, columna o fila por usuario/rol
- **Índices automáticos:** acceso eficiente sin que la aplicación gestione estructuras de búsqueda

Listing 1.1: DDL + DML básico

```

1  -- Crear tabla
2  CREATE TABLE empleados (
3      id          SERIAL PRIMARY KEY,
4      nombre     VARCHAR(100) NOT NULL,
5      depto      VARCHAR(50),
6      salario    NUMERIC(10,2) DEFAULT 0
7  );
8
9  -- Insertar
10 INSERT INTO empleados (nombre, depto, salario)
11 VALUES ('Ana López', 'Ingeniería', 45000.00);
12
13 -- Consultar
14 SELECT nombre, salario
15 FROM     empleados
16 WHERE    depto = 'Ingeniería'
17 ORDER BY salario DESC;

```

Listing 1.2: Tipos de datos comunes

```

1  -- Numéricos
2  id          SERIAL          -- entero autoincremental (alias de INTEGER
   + secuencia)
3  precio     NUMERIC(10,2)    -- exacto: 10 dígitos, 2 decimales
4  ratio      FLOAT           -- punto flotante (impreciso para dinero)
5
6  -- Texto
7  nombre     VARCHAR(100)     -- longitud máxima 100
8  codigo     CHAR(6)          -- longitud fija 6 (rellena con espacios)
9  notas      TEXT             -- longitud ilimitada
10
11 -- Fechas
12 creado     TIMESTAMP DEFAULT NOW()
13 fecha      DATE

```

 FLOAT para dinero

Nunca uses `FLOAT` o `DOUBLE` para valores monetarios. El estándar IEEE 754 de punto flotante introduce errores de representación binaria: $0.1 + 0.2 = 0.30000000000000004$. En operaciones financieras esto acumula errores que producen discrepancias al centavo. Usa siempre `NUMERIC(p,s)` o `DECIMAL(p,s)`, que almacena el número exacto en base 10.

 Preguntas típicas

1. ¿Cuál es la diferencia entre DBMS y RDBMS?
2. ¿Cuándo elegirías NoSQL sobre SQL? Da un ejemplo concreto.
3. ¿Qué ventajas ofrece un DBMS frente a archivos planos para una aplicación con múltiples usuarios?

Capítulo 2

Modelo Relacional

2.1. Conceptos fundamentales

El modelo relacional fue propuesto por E.F. Codd en 1970 en su paper “A Relational Model of Data for Large Shared Data Banks”. Su idea central es representar datos como relaciones matemáticas (tablas) y manipularlos con álgebra relacional. La revolución del modelo relacional fue separar el *qué* almacenar del *cómo* almacenarlo físicamente: el programador describe la estructura lógica y el motor decide la representación física.

Una **relación** no es solo una tabla de datos; tiene propiedades formales: ningún orden de filas es significativo, no puede haber filas duplicadas, y cada celda contiene un valor atómico. Estas propiedades, que parecen restricciones, son precisamente lo que hace al modelo predecible y poderoso: cualquier operación sobre una relación produce otra relación (álgebra cerrada).

Terminología formal vs coloquial

- **Relación** (formal) = **tabla** (coloquial) — conjunto de tuplas del mismo tipo
- **Tupla** (formal) = **fila** / **registro** — instancia de la entidad
- **Atributo** (formal) = **columna** / **campo** — propiedad con dominio de valores válidos
- **Dominio**: conjunto de valores posibles para un atributo (ej. enteros positivos, emails válidos)
- **Esquema**: definición de la estructura — nombre de la relación + atributos + tipos
- **Instancia**: el conjunto de tuplas en un momento dado (los datos reales)
- **Grado**: número de atributos (columnas) de la relación
- **Cardinalidad**: número de tuplas (filas) actuales en la instancia

La independencia física del modelo relacional significa que el motor puede reorganizar datos en disco, cambiar estructuras de almacenamiento o agregar índices sin que las consultas SQL cambien.

2.2. Llaves

Las llaves son el mecanismo fundamental de identidad y relación en el modelo relacional. Sin llaves, no es posible identificar unívocamente un registro ni establecer

referencias entre tablas. La elección de llave primaria es una de las decisiones de diseño más importantes y tiene consecuencias en rendimiento, mantenibilidad y semántica del modelo.

Una **superkey** es cualquier conjunto de atributos que identifica tuplas únicamente. La tabla de empleados con columnas {id, nombre, email}: {id}, {email}, {id, nombre}, {id, email} son todas superkeys. Una **candidate key** es la superkey mínima: quitar cualquier atributo la hace dejar de ser única. En el ejemplo, {id} y {email} son candidate keys; {id, nombre} no lo es porque podría haber dos empleados con el mismo nombre.

- **Primary Key (PK):** candidate key elegida como identificador principal; NOT NULL + UNIQUE garantizados por el DBMS
- **Alternate Key:** candidate key no elegida como PK (se implementa con UNIQUE constraint)
- **Foreign Key (FK):** atributo que referencia la PK de otra tabla; establece la relación entre entidades
- **Surrogate key:** PK artificial generada por el sistema (SERIAL/UUID/IDENTITY) sin significado de negocio
- **Natural key:** PK basada en un atributo del mundo real (RFC, email, CURP, ISBN)
- **Composite key:** PK formada por dos o más atributos; común en tablas de relación M:N

💡 Surrogate vs Natural Key — cuándo usar cada una

Las **surrogate keys** (SERIAL/UUID) son preferibles en la mayoría de los casos: son inmutables (el negocio nunca las cambia), sin significado que pueda volverse incorrecto, eficientes en joins al ser enteros, y simples de generar.

Las **natural keys** tienen sentido cuando el atributo es *verdaderamente* estable y único en el tiempo real, no solo en el modelo de negocio actual. El ISBN de un libro es una buena natural key; el RFC de un empleado parece bueno pero puede cambiar por errores o correcciones. Si hay duda, usa surrogate key.

2.3. Integridad referencial

La integridad referencial garantiza que el grafo de relaciones entre tablas sea consistente: una FK siempre apunta a un registro que existe, o es NULL si la relación es opcional. Sin FK enforced por el DBMS, es trivialmente posible insertar pedidos sin cliente, items sin producto, o comentarios sin usuario — “huérfanos” que corrompen silenciosamente la lógica de negocio.

El DBMS verifica integridad en cada operación: al insertar un hijo, verifica que el padre existe; al borrar un padre, aplica la acción configurada sobre los hijos. Esta verificación tiene un costo mínimo en escritura pero previene categorías enteras de bugs que de otra forma se descubren semanas después en producción.

Las acciones ON DELETE y ON UPDATE controlan qué pasa con los registros hijos cuando el padre cambia o se elimina:

- **CASCADE:** propaga el cambio automáticamente a todos los hijos. Útil para datos que son parte del padre (ítems de una orden).
- **SET NULL:** pone NULL en la FK del hijo. Requiere que la columna admita NULL. Útil para relaciones opcionales (empleado sin departamento).
- **RESTRICT / NO ACTION:** rechaza la operación si existen hijos. Protege contra borrado accidental de datos referenciados.
- **SET DEFAULT:** pone el valor DEFAULT de la columna. Poco usado; requiere que el default tenga sentido semántico.

Listing 2.1: PK, FK e integridad referencial

```

1 CREATE TABLE departamentos (
2     id        SERIAL PRIMARY KEY,
3     nombre    VARCHAR(80) NOT NULL UNIQUE
4 );
5
6 CREATE TABLE empleados (
7     id        SERIAL PRIMARY KEY,
8     nombre    VARCHAR(100) NOT NULL,
9     id_depto  INTEGER REFERENCES departamentos(id)
10             ON DELETE SET NULL
11             ON UPDATE CASCADE
12 );
13
14 -- FK viola integridad si el depto no existe:
15 INSERT INTO empleados (nombre, id_depto) VALUES ('Carlos', 999);
16 -- ERROR: insert or update on table "empleados" violates foreign
    key constraint

```

Listing 2.2: Cardinalidades 1:1, 1:N, M:N

```

1 -- 1:N - un depto tiene muchos empleados (FK en "muchos")
2 -- (ya modelado arriba con id_depto en empleados)
3
4 -- M:N - un empleado puede estar en muchos proyectos
5 CREATE TABLE proyectos (
6     id        SERIAL PRIMARY KEY,
7     nombre    VARCHAR(100) NOT NULL
8 );
9
10 CREATE TABLE empleados_proyectos (
11     id_empleado INTEGER REFERENCES empleados(id)    ON DELETE
12             CASCADE,
13     id_proyecto  INTEGER REFERENCES proyectos(id)    ON DELETE
14             CASCADE,
15     rol          VARCHAR(50),
16     PRIMARY KEY (id_empleado, id_proyecto)
17 );

```



Preguntas típicas

1. ¿Cuál es la diferencia entre Primary Key y Unique Key?
2. ¿Qué sucede con los registros hijos cuando se borra el padre con `ON DELETE CASCADE`?
3. ¿Cuándo preferirías una surrogate key sobre una natural key?

Capítulo 3

SQL Fundamentos

3.1. Sublanguages SQL

SQL (Structured Query Language) es un lenguaje **declarativo**: describes *qué* resultado quieres, no *cómo* calcularlo. El motor de base de datos recibe tu consulta, la analiza, genera un plan de ejecución óptimo y lo ejecuta. Esta abstracción es poderosa: el mismo SQL puede ejecutarse de formas físicas completamente distintas dependiendo de los índices disponibles, el tamaño de las tablas y las estadísticas del optimizador.

SQL se divide en cinco sublanguages según el tipo de operación que realizan. Conocer esta división ayuda a entender qué efecto tiene cada comando y si puede o no revertirse con un ROLLBACK.

Los cinco sublanguages de SQL

- **DDL** (Data Definition Language): define la *estructura* del esquema. CREATE, ALTER, DROP, TRUNCATE, RENAME. En la mayoría de DBMS es auto-commit (no reversible), pero en PostgreSQL el DDL sí es transaccional.
- **DML** (Data Manipulation Language): manipula los *datos* dentro de las estructuras. INSERT, UPDATE, DELETE, MERGE. Siempre dentro de una transacción; reversible con ROLLBACK.
- **DQL** (Data Query Language): consulta datos sin modificarlos. SELECT. El comando más complejo y usado de SQL.
- **DCL** (Data Control Language): controla *permisos* de acceso. GRANT, REVOKE. Define quién puede hacer qué sobre qué objetos.
- **TCL** (Transaction Control Language): gestiona *transacciones*. BEGIN, COMMIT, ROLLBACK, SAVEPOINT, RELEASE SAVEPOINT.

La distinción entre DDL y DML importa porque un DROP TABLE en la mayoría de DBMS no puede revertirse aunque estés dentro de una transacción. PostgreSQL es la excepción: su DDL transaccional permite ROLLBACK de un DROP TABLE.

3.2. DDL — definir estructura

El DDL define el **esquema**: la estructura de las tablas, sus columnas, tipos de datos y las reglas (constraints) que gobiernan qué valores son válidos. Un buen esquema con constraints bien definidos es la primera línea de defensa de integridad de datos: previene datos inválidos a nivel de motor, sin depender de que la aplicación los valide

correctamente.

Los **constraints** son reglas que el DBMS enforcea en cada operación:

- **PRIMARY KEY**: identifica unívocamente cada fila; NOT NULL + UNIQUE implícitos
- **UNIQUE**: no permite valores duplicados en la columna o combinación de columnas
- **NOT NULL**: la columna debe tener valor; nunca puede ser NULL
- **CHECK**: condición booleana que cada valor debe cumplir (ej. `precio > 0`)
- **DEFAULT**: valor que se usa si no se especifica uno en el INSERT
- **FOREIGN KEY**: establece relación con otra tabla; garantiza integridad referencial

ALTER TABLE permite modificar la estructura de una tabla existente sin recrearla. En tablas grandes (millones de filas), algunas operaciones de **ALTER** pueden ser costosas porque requieren reescribir toda la tabla o bloquear escrituras durante la operación. Agregar una columna nullable es barato; agregar una columna NOT NULL sin **DEFAULT** requiere actualizar cada fila existente.

3.3. 🔍 DML + DQL — manipular y consultar

El **SELECT** es el comando más complejo de SQL porque su sintaxis no refleja el orden en que se ejecuta. Se escribe en el orden: **SELECT** → **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **ORDER BY** → **LIMIT**. Pero el motor lo evalúa en un orden completamente distinto, y entender ese orden es crítico para evitar errores y escribir consultas eficientes.

💡 Orden lógico de evaluación de **SELECT**

El motor evalúa en este orden:

1. **FROM + JOIN**: determina el conjunto de filas base
2. **WHERE**: filtra filas del conjunto base (opera sobre columnas reales, no alias)
3. **GROUP BY**: agrupa las filas filtradas
4. **HAVING**: filtra grupos (puede referenciar funciones de agregación)
5. **SELECT**: calcula las expresiones y alias de salida
6. **ORDER BY**: ordena el resultado (puede usar alias del **SELECT**)
7. **LIMIT / OFFSET**: recorta el resultado final

Consecuencia práctica: no puedes usar un alias definido en **SELECT** dentro de **WHERE**, porque **WHERE** se evalúa antes. Pero sí puedes usarlo en **ORDER BY**.

Los **JOINS** combinan filas de dos tablas según una condición. El tipo de **JOIN** determina qué pasa cuando no hay coincidencia:

- **INNER JOIN**: solo filas que coinciden en ambas tablas. Excluye los que no tienen par.

- **LEFT JOIN:** todas las filas de la tabla izquierda; NULL en columnas de la derecha si no hay coincidencia.
- **RIGHT JOIN:** todas las filas de la derecha; NULL en la izquierda si no hay coincidencia. (Equivalente a LEFT JOIN con tablas invertidas.)
- **FULL OUTER JOIN:** todas las filas de ambas tablas; NULL donde no hay coincidencia.
- **CROSS JOIN:** producto cartesiano — cada fila de la izquierda con cada fila de la derecha.

Listing 3.1: DDL: CREATE, ALTER, DROP

```
1 -- Crear tabla con constraints
2 CREATE TABLE clientes (
3     id          SERIAL PRIMARY KEY,
4     email       VARCHAR(200) NOT NULL UNIQUE,
5     nombre      VARCHAR(100) NOT NULL,
6     activo      BOOLEAN DEFAULT TRUE,
7     creado_en   TIMESTAMP DEFAULT NOW()
8 );
9
10 -- Agregar columna
11 ALTER TABLE clientes ADD COLUMN telefono VARCHAR(20);
12
13 -- Agregar constraint después de crear
14 ALTER TABLE clientes ADD CONSTRAINT chk_email CHECK (email LIKE '%@
15 %');
16
17 -- Eliminar columna
18 ALTER TABLE clientes DROP COLUMN telefono;
```

Listing 3.2: DML: INSERT, UPDATE, DELETE

```
1 -- INSERT con RETURNING (PostgreSQL)
2 INSERT INTO clientes (email, nombre)
3 VALUES ('ana@ejemplo.com', 'Ana López')
4 RETURNING id, creado_en;
5
6 -- UPDATE con condición
7 UPDATE clientes
8 SET     activo = FALSE
9 WHERE   creado_en < NOW() - INTERVAL '1 year'
10 AND     activo = TRUE;
11
12 -- DELETE con subquery
13 DELETE FROM clientes
14 WHERE id IN (
15     SELECT id FROM clientes
16     WHERE activo = FALSE
17     ORDER BY creado_en
18     LIMIT 100
19 );
```

Listing 3.3: SELECT con JOINS

```
1  -- INNER JOIN: solo filas que coinciden en ambas tablas
2  SELECT e.nombre, d.nombre AS departamento
3  FROM   empleados e
4  INNER JOIN departamentos d ON e.id_depto = d.id;
5
6  -- LEFT JOIN: todas las filas de la izquierda, NULL si no hay match
7  SELECT e.nombre, d.nombre AS departamento
8  FROM   empleados e
9  LEFT JOIN departamentos d ON e.id_depto = d.id;
10
11 -- GROUP BY + HAVING
12 SELECT  d.nombre, COUNT(e.id) AS total, AVG(e.salario) AS promedio
13 FROM    empleados e
14 JOIN    departamentos d ON e.id_depto = d.id
15 GROUP BY d.nombre
16 HAVING  COUNT(e.id) > 3
17 ORDER BY promedio DESC;
```



Preguntas típicas

1. ¿Cuál es la diferencia entre WHERE y HAVING?
2. ¿Qué diferencia hay entre DELETE y TRUNCATE?
3. Explica los tipos de JOIN: INNER, LEFT, RIGHT, FULL OUTER.

Capítulo 4

SQL Avanzado

4.1. CTEs (Common Table Expressions)

Una CTE es una subconsulta con nombre definida antes del **SELECT** principal usando la cláusula **WITH**. El objetivo principal es legibilidad: en lugar de anidar subconsultas dentro de otras subconsultas (que se vuelven ilegibles rápidamente), se definen como bloques nombrados que se leen de arriba hacia abajo como un programa.

Una CTE puede referenciarse múltiples veces en la misma query, lo que evita duplicación. En PostgreSQL, una CTE por defecto se **materializa**: se ejecuta una vez y el resultado se almacena en memoria. Esto puede ser una ventaja (si se referencia múltiples veces) o una desventaja (si el planificador no puede optimizarla junto con el resto de la query). Con **NOT MATERIALIZED**, le indicas al planificador que trate la CTE como una vista expandida en línea.

Las **CTEs recursivas** usan **WITH RECURSIVE** y se auto-referencian. Son la herramienta estándar en SQL para recorrer estructuras jerárquicas (árboles organizacionales, categorías anidadas, grafos). La recursión termina cuando la parte recursiva no retorna filas.

- CTE regular: alias de subconsulta reutilizable, mejora legibilidad
- CTE recursiva (**WITH RECURSIVE**): auto-referencia para jerarquías y grafos
- **MATERIALIZED**: fuerza ejecución y caché del resultado
- **NOT MATERIALIZED**: permite al planificador optimizar junto con la query externa

4.2. Window Functions

Las window functions son una de las características más poderosas de SQL moderno. Permiten calcular valores en relación a un conjunto de filas (la “ventana”) **sin colapsar el resultado** como hace **GROUP BY**. Cada fila del resultado mantiene su identidad y adicionalmente puede ver valores de sus filas vecinas.

El caso clásico que las necesita: calcular el salario de cada empleado y también el promedio del salario de su departamento en la misma fila. Con **GROUP BY** esto requeriría un **JOIN** adicional; con window functions es una sola expresión.

Anatomía de una Window Function

`función() OVER (PARTITION BY ... ORDER BY ... ROWS/RANGE BETWEEN ...)`

- **función():** ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, FIRST_VALUE, LAST_VALUE, SUM, AVG, COUNT
- **PARTITION BY:** divide las filas en grupos independientes (como GROUP BY pero sin colapsar filas). Cada partición tiene su propia ventana.
- **ORDER BY:** define el orden dentro de cada partición. Obligatorio para funciones que dependen de posición relativa (LAG, LEAD, running totals).
- **ROWS BETWEEN:** delimita exactamente cuántas filas forman la ventana para cada fila actual. UNBOUNDED PRECEDING: desde el inicio de la partición. CURRENT ROW: la fila actual. UNBOUNDED FOLLOWING: hasta el final de la partición.

ROW_NUMBER asigna un número único siempre; **RANK** salta números con empates (1,1,3); **DENSE_RANK** no salta (1,1,2). **LAG(col, n)** devuelve el valor de `col` de `n` filas *antes*; **LEAD(col, n)** de `n` filas *después*.

4.3. EXPLAIN ANALYZE

EXPLAIN y EXPLAIN ANALYZE son las herramientas más importantes para entender por qué una query es lenta y cómo optimizarla. EXPLAIN muestra el plan sin ejecutar la query (estimaciones). EXPLAIN ANALYZE ejecuta la query y muestra tiempos reales junto con las estimaciones, permitiendo comparar.

El plan se muestra como un árbol de nodos. Cada nodo representa una operación; el costo se acumula de hojas a raíz. El nodo más interno (más indentado) se ejecuta primero.

- **Seq Scan:** lee toda la tabla secuencialmente. Normal para tablas pequeñas o cuando se retorna más del 5-10 % de las filas. Preocupante en tablas grandes con filtros selectivos.
- **Index Scan:** usa un índice para localizar filas específicas. Eficiente para alta selectividad (pocos resultados de muchas filas).
- **Bitmap Index Scan + Bitmap Heap Scan:** acumula punteros del índice, los ordena y accede al heap en orden. Más eficiente que múltiples Index Scans.
- **Hash Join:** construye una hash table del lado más pequeño y sondea el grande. Eficiente para joins de tablas medianas/grandes.
- **Nested Loop:** itera el lado externo y busca en el interno. Eficiente cuando el lado interno es pequeño o indexado.
- **Merge Join:** ambas tablas ordenadas; avanza en paralelo. Eficiente cuando los datos ya están ordenados.

- **cost=inicio..total:** unidades de costo abstractas (no segundos). El inicio es el costo antes de la primera fila; el total es el costo de retornar todas las filas.
- **rows:** estimación de filas. Si difiere mucho del **actual rows**, las estadísticas están desactualizadas. Ejecuta **ANALYZE tabla** para actualizarlas.

Listing 4.1: CTEs — regular y recursiva

```

1  -- CTE regular: ventas por región con total acumulado
2  WITH ventas_region AS (
3      SELECT region, SUM(monto) AS total
4      FROM   ventas
5      WHERE  fecha >= '2024-01-01'
6      GROUP BY region
7  )
8  SELECT region, total,
9         ROUND(100.0 * total / SUM(total) OVER (), 2) AS pct
10 FROM   ventas_region
11 ORDER BY total DESC;
12
13 -- CTE recursiva: jerarquía de empleados (árbol)
14 WITH RECURSIVE jerarquia AS (
15     SELECT id, nombre, id_jefe, 0 AS nivel
16     FROM   empleados WHERE id_jefe IS NULL
17
18     UNION ALL
19
20     SELECT e.id, e.nombre, e.id_jefe, j.nivel + 1
21     FROM   empleados e
22     JOIN   jerarquia j ON e.id_jefe = j.id
23 )
24 SELECT nivel, nombre FROM jerarquia ORDER BY nivel, nombre;

```

Listing 4.2: Window Functions

```

1  -- ROW_NUMBER, RANK, DENSE_RANK
2  SELECT nombre, depto, salario,
3         ROW_NUMBER() OVER (PARTITION BY depto ORDER BY salario DESC)
4         AS fila,
5         RANK() OVER (PARTITION BY depto ORDER BY salario DESC)
6         AS rango,
7         DENSE_RANK() OVER (PARTITION BY depto ORDER BY salario DESC)
8         AS rango_denso
9 FROM   empleados;
10
11 -- LAG y LEAD: comparar con fila anterior/siguiente
12 SELECT fecha, ventas,
13        LAG(ventas) OVER (ORDER BY fecha) AS ventas_dia_anterior,
14        ventas - LAG(ventas) OVER (ORDER BY fecha) AS delta
15 FROM   reporte_diario;
16
17 -- Running total
18 SELECT fecha, monto,

```

```
16      SUM(monto) OVER (ORDER BY fecha ROWS UNBOUNDED PRECEDING) AS
      acumulado
17 FROM ventas;
```

Listing 4.3: EXPLAIN ANALYZE

```
1 EXPLAIN ANALYZE
2 SELECT e.nombre, d.nombre
3 FROM empleados e
4 JOIN departamentos d ON e.id_depto = d.id
5 WHERE e.salario > 50000;
6
7 -- Salida típica:
8 -- Hash Join (cost=12.50..34.20 rows=50 width=200)
9 --   -> Seq Scan on empleados
10 --      Filter: (salario > 50000)
11 --   -> Hash
12 --      -> Seq Scan on departamentos
13 -- Si ves "Seq Scan" en tabla grande = considera índice en esa
   columna
```



Preguntas típicas

1. ¿Cuál es la diferencia entre ROW_NUMBER(), RANK() y DENSE_RANK()?
2. ¿Cuándo usarías una CTE recursiva? Da un ejemplo de caso de uso.
3. ¿Cómo interpretas un nodo Seq Scan en EXPLAIN ANALYZE?

Capítulo 5



Modelado UML y Diagramas ER

5.1. Diagrama Entidad-Relación (ER)

El modelo ER es una herramienta conceptual: permite capturar la estructura del dominio de negocio antes de pensar en tablas, tipos de datos o índices. Es independiente del DBMS y sirve como lenguaje común entre desarrolladores y stakeholders. Una vez validado el modelo ER, el paso a SQL (DDL) es casi mecánico.



Elementos del modelo ER

- **Entidad:** objeto del mundo real con existencia propia (rectángulo en Chen)
- **Atributo:** propiedad de la entidad (elipse en Chen / columna en Crow's Foot)
- **Relación:** asociación semántica entre entidades (rombo en Chen, línea en Crow's Foot)
- **Entidad débil:** depende de otra para existir; sin PK propia, usa **llave parcial** combinada con la PK del padre
- **Cardinalidad:** cuántas instancias de una entidad se relacionan con la otra (1:1, 1:N, M:N)

El proceso: identificar entidades → definir atributos → establecer relaciones → asignar cardinalidades → verificar con casos de uso reales.

5.2. Notaciones

Existen varias notaciones para diagramas ER; todas expresan lo mismo con símbolos distintos. La notación Chen es más académica y explícita; Crow's Foot es el estándar en herramientas de producción.

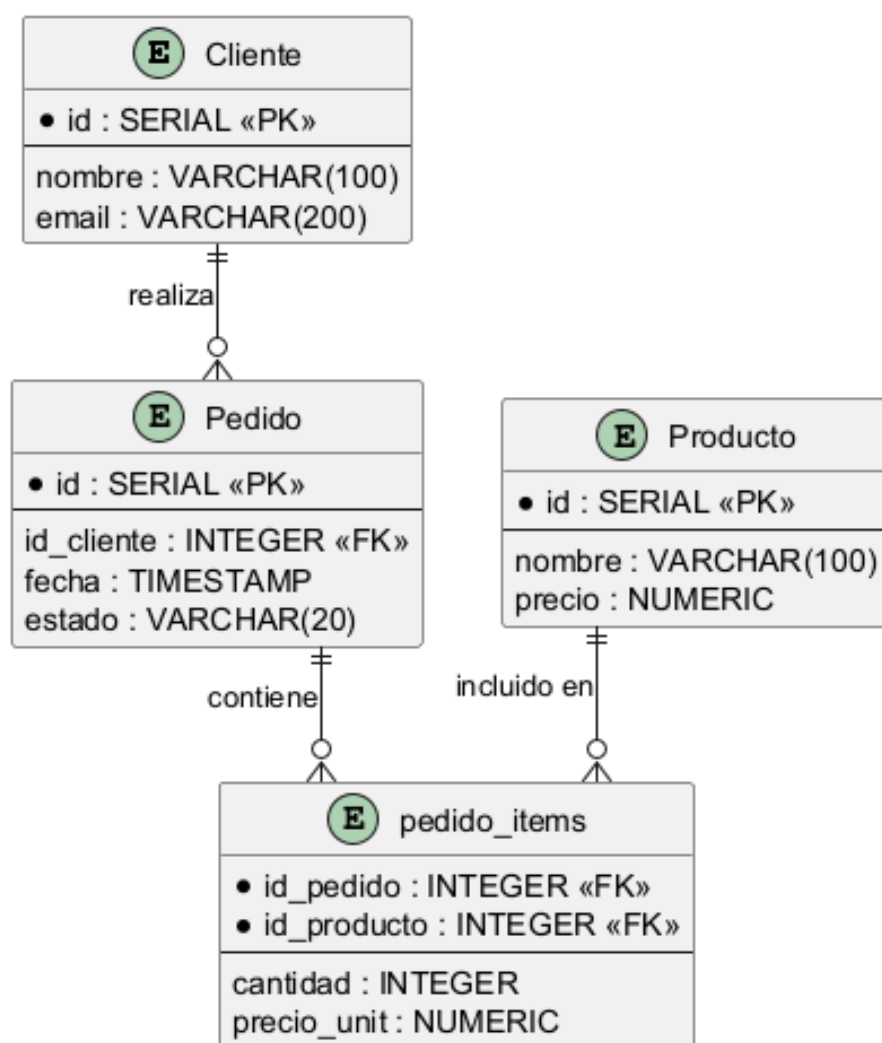
- **Chen:** rectángulos, rombos, elipses. Académica, verbosa, mejor para aprendizaje.
- **Crow's Foot:** líneas con símbolos de cardinalidad en extremos. Estándar en herramientas (Lucidchart, dbdiagram.io, DBeaver).
- **UML Clases:** atributos y métodos en compartimentos. Útil cuando el modelo de datos y el de objetos son lo mismo (ORM).

5.3. Cardinalidad en Crow's Foot

La notación Crow's Foot combina dos elementos en cada extremo de la línea: la **modalidad** (si la participación es obligatoria u opcional) y la **cardinalidad** (uno o muchos). Se leen desde el centro de la relación hacia afuera.

- || exactamente uno (obligatorio)
- |o cero o uno (opcional)
- ||< uno o muchos (obligatorio)
- |o< cero o muchos (opcional)

 Diagrama ER — Cliente, Pedido, Producto



Listing 5.1: DDL derivado del modelo ER

```

1 CREATE TABLE clientes (
2   id      SERIAL PRIMARY KEY,
3   nombre  VARCHAR(100) NOT NULL,
4   email   VARCHAR(200) UNIQUE NOT NULL

```

```
5 );
6
7 CREATE TABLE productos (
8     id          SERIAL PRIMARY KEY,
9     nombre      VARCHAR(100) NOT NULL,
10    precio       NUMERIC(10,2) NOT NULL CHECK (precio >= 0)
11 );
12
13 CREATE TABLE pedidos (
14     id          SERIAL PRIMARY KEY,
15     id_cliente  INTEGER NOT NULL REFERENCES clientes(id),
16     fecha       TIMESTAMP DEFAULT NOW(),
17     estado      VARCHAR(20) DEFAULT 'pendiente'
18 );
19
20 CREATE TABLE pedido_items (
21     id_pedido   INTEGER REFERENCES pedidos(id) ON DELETE CASCADE,
22     id_producto INTEGER REFERENCES productos(id) ON DELETE RESTRICT
23     ,
24     cantidad    INTEGER NOT NULL CHECK (cantidad > 0),
25     precio_unit NUMERIC(10,2) NOT NULL,
26     PRIMARY KEY (id_pedido, id_producto)
27 );
```

Listing 5.2: Consulta con JOIN múltiple

```
1 SELECT p.id AS pedido, c.nombre AS cliente,
2        pr.nombre AS producto, pi.cantidad, pi.precio_unit
3 FROM   pedidos p
4 JOIN   clientes c      ON p.id_cliente = c.id
5 JOIN   pedido_items pi ON pi.id_pedido = p.id
6 JOIN   productos pr   ON pi.id_producto = pr.id
7 WHERE  p.estado = 'pendiente'
8 ORDER BY p.fecha DESC;
```



Preguntas típicas

1. ¿Cuál es la diferencia entre notación Chen y Crow's Foot?
2. ¿Cómo modelas una relación M:N en SQL?
3. ¿Qué es una entidad débil? Da un ejemplo.

Capítulo 6

Normalización

6.1. ¿Por qué normalizar?

La normalización es un proceso sistemático de diseño de esquemas relacionales que elimina redundancia y las anomalías que esta produce. Fue formalizada por E.F. Codd junto con el modelo relacional y se basa en el concepto de **dependencias funcionales**: cómo los valores de ciertos atributos determinan los valores de otros.

El problema de la redundancia es sutil: la misma información almacenada en múltiples lugares no es solo un desperdicio de espacio, sino una fuente de inconsistencia. Cuando el mismo dato está en muchas filas, cualquier actualización debe propagarse a todas ellas. Si falla a la mitad, la BD queda en estado inconsistente sin que el DBMS lo detecte (porque cada fila individualmente es válida).

Las tres anomalías de la desnormalización

Considera una tabla `empleado_depto(id_emp, nombre_emp, id_depto, nombre_depto, salario)` donde `nombre_depto` depende de `id_depto`, no de `id_emp`:

- **Anomalía de inserción:** es imposible registrar un departamento nuevo sin tener al menos un empleado en él, porque `nombre_depto` solo existe junto a `id_emp`.
- **Anomalía de actualización:** si el departamento “Ingeniería” cambia de nombre a “Tecnología”, hay que actualizar *cada fila* de cada empleado de ese depto. Si se actualiza solo parte, la BD tiene dos nombres distintos para el mismo departamento.
- **Anomalía de borrado:** si el último empleado del departamento “Marketing” deja la empresa y se borra su fila, se pierde toda la información del departamento “Marketing” junto con ella.

La solución a las tres anomalías es la misma: separar cada entidad en su propia tabla y usar FK para relacionarlas.

6.2. Formas normales

Las formas normales son criterios progresivos de diseño, donde cada nivel incluye al anterior y agrega una restricción más específica. No son reglas arbitrarias: cada forma normal elimina una categoría específica de anomalía. En la práctica, 3NF es el objetivo estándar para bases de datos transaccionales (OLTP).

- **1NF (Primera Forma Normal):** Cada celda contiene un valor atómico (no listas, no arrays, no múltiples valores separados por coma). No hay filas duplicadas. La tabla tiene una PK definida. Violación clásica: columna **telefonos** con valor “555-1234, 555-5678”.
- **2NF (Segunda Forma Normal):** En 1NF + cada atributo no-clave depende de *toda* la PK compuesta, no solo de una parte. Solo aplica cuando la PK está formada por múltiples columnas (PK compuesta). Violación: tabla con PK=(id_pedido, id_producto) y una columna **nombre_producto** que depende solo de **id_producto**.
- **3NF (Tercera Forma Normal):** En 2NF + ningún atributo no-clave depende transitivamente de otro atributo no-clave. Violación: tabla con **id_depto** → **nombre_depto**, donde ambos son atributos no-clave. El atributo **nombre_depto** depende de **id_depto**, no directamente de la PK.
- **BCNF (Boyce-Codd Normal Form):** Más estricta que 3NF: todo determinante debe ser una superkey. Cubre casos extremos donde 3NF no es suficiente (relaciones con múltiples candidate keys superpuestas).

💡 ¿Cuándo desnormalizar intencionalmente?

La desnormalización sacrifica integridad por rendimiento y es válida en contextos específicos:

- **Data Warehouses (OLAP):** star schema y snowflake schema desnormalizan intencionalmente para evitar joins costosos en queries de millones de filas.
- **Cachés de lectura:** tablas desnormalizadas para reportes frecuentes que se actualizan por batch.
- **Campos calculados persistidos:** cuando recalcular en cada consulta es más costoso que mantener la redundancia con un trigger.

Desnormalizar siempre debe ser una decisión consciente y documentada, nunca por accidente.

6.3. Dependencias funcionales

Una dependencia funcional $X \rightarrow Y$ significa: conocer el valor de X determina unívocamente el valor de Y . Es la herramienta matemática para razonar sobre la normalización: identificar dependencias funcionales en un esquema es equivalente a identificar qué forma normal tiene.

Ejemplos concretos:

- $\text{id_empleado} \rightarrow \text{nombre_empleado}$: saber el id determina el nombre (dependencia total de la PK — correcto en 2NF)
- $\text{id_producto} \rightarrow \text{nombre_producto}$ dentro de PK=(id_pedido, id_producto): dependencia **parcial** — viola 2NF
- $\text{id_depto} \rightarrow \text{nombre_depto}$ donde **id_depto** no es clave: dependencia **transitiva** — viola 3NF

Si $PK = \{A, B\}$ y existe $A \rightarrow C$ (C no depende de B) $\rightarrow$ dependencia **parcial** $\rightarrow$ viola 2NF.

Si existe no-clave $\rightarrow$ no-clave (por ejemplo $id_depto \rightarrow nombre_depto$) $\rightarrow$ dependencia **transitiva** $\rightarrow$ viola 3NF.

Listing 6.1: Tabla que viola 2NF y su corrección

```

1  -- VIOLA 2NF: PK=(id_pedido, id_producto) pero nombre_producto
2  --           depende solo de id_producto (dependencia parcial)
3  CREATE TABLE pedido_items_malo (
4      id_pedido      INTEGER,
5      id_producto    INTEGER,
6      nombre_producto VARCHAR(100),
7      cantidad       INTEGER,
8      PRIMARY KEY (id_pedido, id_producto)
9  );
10
11 -- CORRECTO: separar producto a su propia tabla
12 CREATE TABLE productos (
13     id      SERIAL PRIMARY KEY,
14     nombre  VARCHAR(100) NOT NULL
15 );
16
17 CREATE TABLE pedido_items (
18     id_pedido  INTEGER,
19     id_producto INTEGER REFERENCES productos(id),
20     cantidad   INTEGER NOT NULL,
21     PRIMARY KEY (id_pedido, id_producto)
22 );

```

Listing 6.2: Tabla que viola 3NF y su corrección

```

1  -- VIOLA 3NF: id_depto -> nombre_depto (dependencia transitiva)
2  CREATE TABLE empleados_malo (
3      id_empleado  SERIAL PRIMARY KEY,
4      nombre       VARCHAR(100),
5      id_depto     INTEGER,
6      nombre_depto VARCHAR(80)
7  );
8
9  -- CORRECTO: tabla separada para departamentos
10 CREATE TABLE departamentos (
11     id      SERIAL PRIMARY KEY,
12     nombre  VARCHAR(80) NOT NULL
13 );
14
15 CREATE TABLE empleados (
16     id      SERIAL PRIMARY KEY,
17     nombre  VARCHAR(100),
18     id_depto INTEGER REFERENCES departamentos(id)
19 );

```

**Preguntas típicas**

1. ¿Cuál es la diferencia entre 2NF y 3NF?
2. ¿Cuándo desnormalizarías intencionalmente una tabla?
3. Identifica la forma normal: (orden_id, producto_id, categoria_producto, cantidad) con PK=(orden_id, producto_id).

Capítulo 7

Índices

7.1. ¿Qué es un índice?

Un índice es una estructura de datos auxiliar que mantiene una copia ordenada de uno o más atributos de una tabla, con punteros a las filas originales en el heap (la tabla principal). La analogía del índice de un libro es precisa: en lugar de leer todo el libro para encontrar un tema (Seq Scan), vas directamente a la página referenciada (Index Scan).

Sin índice, el motor debe leer *cada fila* de la tabla para encontrar las que cumplen el filtro — esto es $O(n)$. Con un índice B-tree, la búsqueda es $O(\log n)$ para localizar la primera fila, y luego secuencial para las siguientes. En una tabla de 10 millones de filas, la diferencia entre Seq Scan e Index Scan puede ser de 8 segundos vs 0.3 milisegundos.

Estructura interna de un índice B-tree

Un B-tree (árbol B) es un árbol balanceado de búsqueda con estas propiedades:

- **Nodos internos:** contienen claves y punteros a nodos hijo. Guían la búsqueda de $O(\log n)$.
- **Nodos hoja:** contienen las claves ordenadas y el puntero al heap (tid: page + offset). Son el nivel que finalmente referencia los datos.
- **Balanceado:** todas las hojas están a la misma profundidad. Un árbol con 1 millón de entradas tiene solo 20 niveles.
- **Soporte de rangos:** las hojas están enlazadas entre sí, permitiendo escaneos de rango eficientes (BETWEEN, >, <).

Costo real de mantener un índice: cada INSERT, UPDATE o DELETE debe actualizar todas las estructuras de índice de la tabla. En una tabla con 5 índices, cada escritura genera 5 operaciones adicionales de árbol. En tablas con escritura masiva, esto puede ser el cuello de botella.

7.2. Tipos de índices en PostgreSQL

PostgreSQL ofrece varios tipos de índices porque no existe una estructura óptima para todos los patrones de consulta. Cada tipo está optimizado para un conjunto específico de operadores y tipos de datos. Elegir el tipo incorrecto puede hacer que el índice sea ignorado por el planificador.

- **B-tree (default):** árbol balanceado. Soporta: =, <, >, <=, >=, BETWEEN, LIKE 'prefix%', IS NULL, IN. El 90 % de los casos. Si no sabes qué tipo usar, usa

B-tree.

- **Hash:** tabla hash. Solo soporta =. Ligeramente más rápido que B-tree para igualdad exacta, pero no soporta rangos y no se replica en versiones antiguas de PG. Poco usado.
- **GIN** (Generalized Inverted Index): índice invertido — mapea elementos a las filas que los contienen. Ideal para: arrays (`@>`, `&&`), JSONB (`@>`, `?`), full-text search (`@@`). Lento en escritura (reconstruye la estructura), muy rápido en consultas.
- **GiST** (Generalized Search Tree): framework extensible para tipos de datos no estándar. Usado para: geometría/geo (PostGIS), rangos (`tsrange`, `daterange`), full-text. Más versátil que GIN, algo más lento en búsqueda.
- **BRIN** (Block Range Index): almacena rangos mínimo-máximo por bloque físico. Muy compacto (tamaño ínfimo comparado con B-tree). Eficiente solo cuando los datos están físicamente ordenados por la columna indexada (series de tiempo insertadas en orden, logs).

7.3. 💡 Cuándo crear (y no crear) un índice

La **selectividad** es el factor más importante: un índice es útil cuando filtra una fracción pequeña de las filas. Si una condición retorna el 50 % de las filas, un Seq Scan puede ser más eficiente que un Index Scan porque lee el heap secuencialmente (más amigable con el cache de disco). El planificador de PostgreSQL conoce estadísticas de distribución de valores y decide automáticamente cuándo usar el índice.

💡 Reglas prácticas para índices

Crea índice en:

- Columnas en `WHERE` con alta selectividad (pocos resultados de muchas filas)
- Columnas de FK — PostgreSQL *no* las indexa automáticamente; MySQL sí
- Columnas en `JOIN ON` y `ORDER BY` en queries frecuentes con tablas grandes
- Columnas de búsqueda full-text (índice GIN con `to_tsvector`)

No crees índice en:

- Tablas con menos de 1,000 filas — el planificador preferirá Seq Scan de todas formas
- Columnas con cardinalidad muy baja: booleans, `status` con 3 valores posibles
- Columnas que se actualizan muy frecuentemente — el costo de mantenimiento supera la ganancia de lectura
- Columnas raramente usadas en `WHERE` — el índice ocupa espacio y ralentiza escrituras sin beneficio

Listing 7.1: Crear índices B-tree, parcial y compuesto

```
1 -- Índice simple (B-tree por defecto)
2 CREATE INDEX idx_empleados_depto ON empleados(id_depto);
```

```

3
4 -- Índice compuesto
5 CREATE INDEX idx_pedidos_cliente_fecha ON pedidos(id_cliente, fecha
  DESC);
6
7 -- Índice parcial: solo indexa subset de filas
8 CREATE INDEX idx_pedidos_pendientes ON pedidos(fecha)
9 WHERE estado = 'pendiente';
10
11 -- Índice UNIQUE
12 CREATE UNIQUE INDEX idx_clientes_email ON clientes(email);
13
14 -- Crear índice SIN bloquear escrituras (producción)
15 CREATE INDEX CONCURRENTLY idx_ventas_fecha ON ventas(fecha);

```

Listing 7.2: EXPLAIN ANALYZE — verificar uso de índice

```

1 -- Sin índice: Seq Scan
2 EXPLAIN ANALYZE
3 SELECT * FROM empleados WHERE salario > 80000;
4 -- Seq Scan on empleados (cost=0..520 rows=15)
5 -- Execution time: 8.4 ms
6
7 -- Crear índice
8 CREATE INDEX idx_empleados_salario ON empleados(salario);
9
10 -- Con índice: Index Scan
11 EXPLAIN ANALYZE
12 SELECT * FROM empleados WHERE salario > 80000;
13 -- Index Scan using idx_empleados_salario
14 -- Execution time: 0.3 ms
15
16 -- GIN para búsqueda en JSONB
17 CREATE INDEX idx_datos_gin ON tabla USING GIN (datos_jsonb);
18 SELECT * FROM tabla WHERE datos_jsonb @> '{"activo": true}';

```

Listing 7.3: Índices y FK en PostgreSQL

```

1 -- PostgreSQL NO crea índice automático en FK
2 -- Sin este índice, DELETE en tabla padre hace Seq Scan en hijos
3 CREATE INDEX idx_empleados_id_depto ON empleados(id_depto);
4 CREATE INDEX idx_pedidos_id_cliente ON pedidos(id_cliente);
5
6 -- Ver índices existentes
7 SELECT indexname, indexdef
8 FROM pg_indexes
9 WHERE tablename = 'empleados';

```

**Preguntas típicas**

1. ¿Cuándo usarías un índice GIN en lugar de B-tree?
2. ¿Por qué los índices ralentizan las escrituras?
3. ¿Qué es un índice parcial y cuándo tiene sentido usarlo?

Capítulo 8

Transacciones y Control de Concurrency

8.1. ACID

Una transacción es una secuencia de operaciones que el DBMS trata como una unidad indivisible: o todas se completan exitosamente, o ninguna tiene efecto. Las propiedades ACID son las garantías que hacen a las transacciones confiables incluso en presencia de fallos del sistema, cortes de energía o acceso concurrente de miles de usuarios simultáneos.

ACID no es magia: es el resultado de mecanismos concretos implementados por el DBMS. Atomicidad usa undo logs; Durabilidad usa WAL (Write-Ahead Log); Aislamiento usa locks o MVCC; Consistencia es responsabilidad compartida entre el DBMS (constraints) y la aplicación (lógica de negocio).

ACID explicado con transferencia bancaria

Transferir \$1,000 de la cuenta A a la cuenta B requiere dos operaciones: (1) restar \$1,000 de A y (2) sumar \$1,000 a B.

- **Atomicidad:** si el servidor se cae entre (1) y (2), al reiniciar el DBMS deshace la operación (1) usando el undo log. El dinero no desaparece “a la mitad”. O ambas operaciones ocurren, o ninguna.
- **Consistencia:** la suma total del dinero en el sistema es idéntica antes y después. Ningún constraint se viola durante ni después de la transacción. Si A no tiene saldo suficiente y hay un CHECK constraint, la transacción falla completa.
- **Aislamiento:** si otra transacción consulta el saldo de A mientras la transferencia está en curso, no ve el estado intermedio (A descontado, B sin acreditar). Ve o el estado antes de la transferencia, o el estado después — nunca el estado a medias.
- **Durabilidad:** una vez que el COMMIT regresa con éxito, los cambios están en el WAL en disco. Aunque el servidor se apague inmediatamente, al reiniciar el DBMS replaya el WAL y los datos están ahí.

8.2. Niveles de aislamiento

El aislamiento perfecto (serializable) implicaría que las transacciones se ejecuten como si fueran estrictamente secuenciales. Esto sería correcto pero extremadamente lento: no habría concurrencia real. Los niveles de aislamiento son el compromiso entre correctitud y rendimiento: mayor aislamiento = más correctitud, menos concurrencia.

El estándar SQL define cuatro anomalías que los niveles de aislamiento previenen o permiten:

- **Dirty Read:** una transacción lee datos de otra que aún no hizo COMMIT. Si la segunda hace ROLLBACK, la primera leyó datos que nunca existieron.
- **Non-Repeatable Read:** una transacción lee la misma fila dos veces y obtiene valores distintos porque otra transacción la modificó entre lecturas.
- **Phantom Read:** una transacción ejecuta la misma query dos veces y obtiene filas distintas porque otra transacción insertó o eliminó filas que cumplen el filtro.
- **Serialization Anomaly:** el resultado de ejecutar transacciones concurrentemente no es posible obtenerlo con ningún orden serial de las mismas transacciones.

Los cuatro niveles de aislamiento en PostgreSQL:

- **Read Uncommitted:** en PostgreSQL se trata igual que Read Committed (no existe dirty read en PG).
- **Read Committed (default PG):** cada sentencia ve solo los datos que estaban confirmados al momento de iniciar esa sentencia. Permite non-repeatable reads.
- **Repeatable Read:** la transacción ve un snapshot fijo del momento en que inició. Mismo valor al releer la misma fila. Previene dirty y non-repeatable reads. En PG también previene phantom reads.
- **Serializable:** garantía máxima. PG detecta ciclos de dependencia entre transacciones concurrentes y aborta las que crearían anomalías de serialización.

¿Qué nivel usar en la práctica?

Read Committed (default) cubre el 95 % de los casos de aplicaciones web. Usa **Repeatable Read** cuando una transacción hace múltiples lecturas del mismo dato y la inconsistencia entre ellas causaría lógica incorrecta (reportes financieros, cálculos que dependen de varios SELECTs). Usa **Serializable** solo cuando la correctitud lógica de la operación dependa de ausencia total de interferencia concurrente. Tiene overhead de detección de conflictos y puede generar más errores que requieren retry.

8.3. Locks y Deadlocks

PostgreSQL usa MVCC (Multi-Version Concurrency Control) como mecanismo principal de concurrencia. En MVCC, en lugar de bloquear filas para lectura, el motor mantiene múltiples versiones de cada fila. Cada transacción ve un snapshot consistente

de la BD al momento de su inicio, sin necesidad de locks de lectura. Esto elimina el clásico problema de lecturas que bloquean escrituras y viceversa.

Sin embargo, dos escrituras concurrentes sobre la misma fila sí requieren coordinación. PostgreSQL usa locks a nivel de fila para escrituras: la segunda escritura espera a que la primera haga COMMIT o ROLLBACK.

Un **deadlock** ocurre cuando dos o más transacciones forman un ciclo de espera: T1 espera un lock que tiene T2, y T2 espera un lock que tiene T1. Ninguna puede avanzar indefinidamente. PostgreSQL detecta estos ciclos periódicamente y aborta la transacción más joven (menor costo de rollback), retornando un error al cliente para que reintente.

⚠️ Cómo prevenir deadlocks

La causa raíz de casi todos los deadlocks es acceder a recursos (filas, tablas) en órdenes distintos en transacciones diferentes. Solución: establecer y documentar un **orden canónico** de acceso a recursos en toda la aplicación. Si T1 y T2 siempre adquieren locks en el orden Recurso A → Recurso B, el deadlock es matemáticamente imposible. Además: mantener las transacciones lo más cortas posible y nunca esperar input del usuario dentro de una transacción abierta.

Listing 8.1: BEGIN, COMMIT, ROLLBACK, SAVEPOINT

```

1 BEGIN;
2
3 UPDATE cuentas SET saldo = saldo - 1000 WHERE id = 1;
4 UPDATE cuentas SET saldo = saldo + 1000 WHERE id = 2;
5
6 -- Si algo falla:
7 ROLLBACK;
8 -- Si todo OK:
9 COMMIT;
10
11 -- SAVEPOINT: rollback parcial
12 BEGIN;
13 UPDATE inventario SET stock = stock - 5 WHERE producto_id = 10;
14 SAVEPOINT antes_descuento;
15 UPDATE precios SET precio = precio * 0.9 WHERE producto_id = 10;
16 ROLLBACK TO SAVEPOINT antes_descuento;
17 COMMIT;

```

Listing 8.2: SELECT FOR UPDATE y nivel de aislamiento

```

1 -- Bloquear fila para actualización (evita race condition)
2 BEGIN;
3 SELECT saldo FROM cuentas WHERE id = 1 FOR UPDATE;
4 UPDATE cuentas SET saldo = saldo - 500 WHERE id = 1;
5 COMMIT;
6
7 -- Cambiar nivel de aislamiento
8 BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
9 SELECT total FROM reporte WHERE mes = '2024-01';

```

```
10 COMMIT;
```

Listing 8.3: Detectar y matar conexiones colgadas

```
1  -- Ver transacciones activas hace más de 5 minutos
2  SELECT pid, username, state, query_start,
3         NOW() - query_start AS duracion,
4         LEFT(query, 80) AS query_corta
5  FROM   pg_stat_activity
6  WHERE  state != 'idle'
7         AND query_start < NOW() - INTERVAL '5 minutes';
8
9  -- Terminar transacción colgada
10 SELECT pg_terminate_backend(12345);
```



Preguntas típicas

1. Explica ACID con un ejemplo de transferencia bancaria.
2. ¿Qué es un deadlock y cómo lo detecta/resuelve PostgreSQL?
3. ¿Cuál es la diferencia entre Read Committed y Repeatable Read?

Capítulo 9

PostgreSQL — Características Avanzadas

9.1. Por qué PostgreSQL

PostgreSQL es el RDBMS open source más completo y conforme al estándar SQL. A diferencia de MySQL, es extensible por diseño: cualquiera puede agregar tipos de dato, operadores, funciones y métodos de acceso mediante extensiones. Su MVCC robusto, soporte nativo de JSON, y herramientas de administración maduras lo hacen la primera opción para aplicaciones nuevas.

Arquitectura de PostgreSQL

- **Proceso postmaster:** proceso padre que gestiona conexiones; cada cliente obtiene su propio proceso hijo
 - **WAL (Write-Ahead Log):** todos los cambios se escriben al log antes de modificar los datos; garantiza durabilidad y permite PITR
 - **MVCC:** cada fila tiene versiones; lecturas ven el snapshot de su transacción sin bloquear escrituras
 - **Shared Buffer Cache:** pool de memoria compartida para páginas de disco; `shared_buffers` controla su tamaño (típico: 25 % de RAM)
 - **Autovacuum:** proceso en background que recicla versiones muertas de filas (generadas por MVCC)
-
- **JSONB:** JSON binario indexable (vs JSON texto, que no puede usar índices GIN)
 - **Extensiones:** PostGIS (geo), `pg_trgm` (trigrams/búsqueda difusa), `uuid-oss`, `pgcrypto`
 - **VACUUM:** recupera espacio de filas muertas que deja el MVCC
 - **Roles:** sistema de permisos basado en roles heredables, no usuarios individuales
 - **Particionamiento nativo:** RANGE, LIST, HASH (PG 10+)

9.2. JSONB

JSONB almacena JSON en formato binario descompuesto: lo parsea al insertar y guarda la estructura interna. Esto permite índices GIN y operadores de búsqueda que JSON texto no soporta. Usar JSONB cuando el dato es inherentemente semiestructurado; no como sustituto de columnas relacionales bien definidas.

- -> devuelve JSON (objeto/array) — útil para extraer subobjetos
- ->> devuelve texto — para comparaciones con `WHERE`
- @> contiene (compatible con índice GIN) — operador más importante
- ? tiene la clave — verificar existencia sin importar el valor

9.3. VACUUM y mantenimiento

El MVCC de PostgreSQL crea *versiones muertas* de filas al hacer `UPDATE` o `DELETE`: la fila antigua queda en el heap pero invisible para nuevas transacciones. `VACUUM` recupera ese espacio. Sin `VACUUM` periódico, la tabla crece indefinidamente (table bloat) y el planificador trabaja con estadísticas desactualizadas.

VACUUM FULL en producción

`VACUUM FULL` compacta la tabla reescribiéndola desde cero, liberando el espacio al SO. El problema: bloquea la tabla con un lock exclusivo durante todo el proceso. En producción, prefiere `VACUUM` regular (que solo marca espacio como reutilizable sin bloquear) y reserva `VACUUM FULL` para ventanas de mantenimiento planificadas.

Listing 9.1: JSONB — consultas y operadores

```

1 CREATE TABLE eventos (
2     id        SERIAL PRIMARY KEY,
3     datos     JSONB NOT NULL,
4     creado   TIMESTAMP DEFAULT NOW()
5 );
6
7 INSERT INTO eventos (datos) VALUES
8     ('{"tipo": "login", "usuario": "ana", "ip": "192.168.1.1"}'),
9     ('{"tipo": "compra", "usuario": "carlos", "monto": 350.00}');
10
11 -- Acceder a campos
12 SELECT datos->>'usuario' AS usuario, datos->>'tipo' AS tipo
13 FROM eventos;
14
15 -- Filtrar con @> (usa índice GIN)
16 CREATE INDEX idx_eventos_gin ON eventos USING GIN(datos);
17 SELECT * FROM eventos WHERE datos @> '{"tipo": "login"}';
18
19 -- Actualizar campo en JSONB
20 UPDATE eventos
21 SET datos = datos || '{"procesado": true}'::jsonb
22 WHERE datos->>'tipo' = 'compra';

```

Listing 9.2: Roles, permisos y VACUUM

```

1 -- Crear rol de solo lectura
2 CREATE ROLE readonly;

```

```

3 GRANT CONNECT ON DATABASE mibd TO readonly;
4 GRANT USAGE ON SCHEMA public TO readonly;
5 GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;
6
7 -- Crear usuario con ese rol
8 CREATE USER reportes_user WITH PASSWORD 'segura123';
9 GRANT readonly TO reportes_user;
10
11 -- VACUUM manual
12 VACUUM ANALYZE empleados;
13
14 -- Ver tablas que necesitan vacuum
15 SELECT relname, n_dead_tup, last_vacuum
16 FROM   pg_stat_user_tables
17 WHERE  n_dead_tup > 10000
18 ORDER BY n_dead_tup DESC;

```

Listing 9.3: Particionamiento nativo en PostgreSQL

```

1 -- Tabla particionada por rango de fecha
2 CREATE TABLE ventas (
3     id          SERIAL,
4     fecha       DATE NOT NULL,
5     monto       NUMERIC(10,2),
6     region      VARCHAR(50),
7     PRIMARY KEY (id, fecha)
8 ) PARTITION BY RANGE (fecha);
9
10 -- Crear particiones
11 CREATE TABLE ventas_2023 PARTITION OF ventas
12     FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
13
14 CREATE TABLE ventas_2024 PARTITION OF ventas
15     FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
16
17 -- PostgreSQL enruta automáticamente al insertar
18 INSERT INTO ventas (fecha, monto, region)
19 VALUES ('2024-06-15', 5000.00, 'Norte');

```

Preguntas típicas

1. ¿Cuál es la diferencia entre JSON y JSONB en PostgreSQL?
2. ¿Qué hace VACUUM y por qué es necesario en PostgreSQL?
3. ¿Cómo implementarías seguridad de solo lectura para un usuario de reportes?

Capítulo 10

MySQL / MariaDB

10.1. InnoDB vs MyISAM

MySQL históricamente separaba el servidor del motor de almacenamiento. InnoDB es el motor transaccional; MyISAM es el motor legado sin transacciones. Desde MySQL 5.5 (2010), InnoDB es el motor por defecto y la elección correcta en la práctica totalidad de los casos.

InnoDB vs MyISAM: diferencias clave

- **InnoDB:** ACID completo, FK enforced, row-level locking, MVCC, crash recovery automático
- **MyISAM:** sin transacciones, sin FK, table-level lock (bloquea toda la tabla por escritura), crash recovery manual
- **Rendimiento:** MyISAM puede ser más rápido en lecturas puras de bulk (sin concurrencia), pero InnoDB supera en cualquier escenario real con escrituras concurrentes
- **Regla práctica:** usa siempre InnoDB. MyISAM solo aparece en sistemas heredados o tablas temporales de solo lectura con volumen extremo.

10.2. Diferencias clave con PostgreSQL

MySQL y PostgreSQL comparten el estándar SQL pero difieren en sintaxis, semántica y características. Conocer las diferencias es crítico al migrar entre motores o trabajar en proyectos mixtos.

- `AUTO_INCREMENT` (MySQL) vs `SERIAL` / `GENERATED ALWAYS AS IDENTITY` (PG)
- MySQL no tiene `RETURNING` en `INSERT/UPDATE`; PostgreSQL sí
- `GROUP BY` en MySQL es más permisivo con `ONLY_FULL_GROUP_BY` desactivado (comportamiento peligroso)
- MySQL usa `SHOW PROCESSLIST`, `SHOW STATUS` para monitoreo; PG usa `pg_stat_activity`
- Replicación master-replica nativa vía binlog en MySQL; PG usa WAL streaming replication
- MySQL tiene `ON DUPLICATE KEY UPDATE` para upserts; PG usa `INSERT ... ON CONFLICT`

10.3. Particionamiento

El particionamiento divide físicamente una tabla lógica en subtablas según un criterio. Mejora rendimiento cuando las queries filtran por la columna de partición (partition pruning): el motor lee solo las particiones relevantes.

Partition Pruning

Si la columna de partición está en el `WHERE`, MySQL descarta particiones automáticamente. `EXPLAIN` mostrará `partitions: p2023` en lugar de leer toda la tabla. Sin la columna de partición en el filtro, se leen *todas* las particiones — peor que sin particionamiento.

Listing 10.1: Sintaxis MySQL — diferencias con PostgreSQL

```

1  -- AUTO_INCREMENT (MySQL) vs SERIAL (PostgreSQL)
2  CREATE TABLE productos (
3      id          INT AUTO_INCREMENT PRIMARY KEY,
4      nombre     VARCHAR(100) NOT NULL,
5      precio     DECIMAL(10,2)
6  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
7
8  -- UPSERT en MySQL
9  INSERT INTO clientes (email, nombre, visitas)
10 VALUES ('ana@ej.com', 'Ana', 1)
11 ON DUPLICATE KEY UPDATE visitas = visitas + 1;
12
13 -- Ver motor de cada tabla
14 SELECT table_name, engine
15 FROM   information_schema.tables
16 WHERE  table_schema = 'mi_base';

```

Listing 10.2: Particionamiento por RANGE en MySQL

```

1  CREATE TABLE ventas (
2      id          INT AUTO_INCREMENT,
3      fecha       DATE NOT NULL,
4      monto       DECIMAL(10,2),
5      PRIMARY KEY (id, fecha)
6  )
7  PARTITION BY RANGE (YEAR(fecha)) (
8      PARTITION p2022 VALUES LESS THAN (2023),
9      PARTITION p2023 VALUES LESS THAN (2024),
10     PARTITION p2024 VALUES LESS THAN (2025),
11     PARTITION pfuture VALUES LESS THAN MAXVALUE
12 );
13
14 -- Partition pruning automático
15 EXPLAIN SELECT * FROM ventas
16 WHERE fecha BETWEEN '2023-01-01' AND '2023-12-31';

```

Listing 10.3: Monitoreo y replicación básica

```
1  -- Ver procesos activos
2  SHOW PROCESSLIST;
3
4  -- Ver estado del servidor
5  SHOW STATUS LIKE 'Threads%';
6  SHOW STATUS LIKE 'Innodb_row_lock%';
7
8  -- Estado de replicación (en esclavo)
9  SHOW SLAVE STATUS\G
10
11 -- Variables de configuración
12 SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
13 SHOW VARIABLES LIKE 'max_connections';
```



Preguntas típicas

1. ¿Cuáles son las diferencias principales entre InnoDB y MyISAM?
2. ¿Cómo implementarías un UPSERT en MySQL?
3. ¿Qué es el partition pruning y cuándo beneficia al rendimiento?

Capítulo 11

Bases de Datos NoSQL

11.1. ¿Cuándo NoSQL?

“NoSQL” (Not Only SQL) es un término paraguas para sistemas de almacenamiento que no siguen el modelo relacional estricto. No significa que SQL sea malo — significa que existen patrones de datos y acceso para los que el modelo relacional no es la herramienta más adecuada. La elección correcta requiere entender el *modelo de datos*, los *patrones de acceso* y las *garantías* que el sistema necesita.

El modelo relacional exige definir el esquema antes de insertar datos, enforces integridad referencial, y usa joins para combinar datos de múltiples tablas. Estas características son ventajas para datos estructurados con relaciones complejas y transacciones ACID. Pero se convierten en fricción cuando: el esquema cambia frecuentemente durante el desarrollo, cada registro tiene atributos diferentes, o se necesita escalar escrituras horizontalmente a través de múltiples nodos sin coordinación.

¿Cuándo considerar NoSQL?

- **Esquema variable:** un catálogo de e-commerce donde cada producto (laptop, zapato, alimento) tiene atributos completamente distintos. Un modelo documental almacena cada producto como un documento JSON con su propia estructura.
- **Escalado horizontal de escritura:** sistemas que reciben millones de escrituras por segundo (IoT, telemetría, logs). Cassandra puede distribuir escrituras entre cientos de nodos sin coordinación.
- **Caché de baja latencia:** sesiones de usuario, resultados de queries costosas, contadores en tiempo real. Redis sirve datos desde RAM con latencia de microsegundos.
- **Relaciones como dato principal:** redes sociales, grafos de conocimiento. En Neo4j, traversar 5 niveles de “amigos de amigos” es una operación nativa; en SQL requeriría CTEs recursivas costosas.

Lo que NoSQL generalmente *no* resuelve mejor que SQL: transacciones complejas multi-entidad, queries ad-hoc arbitrarias, integridad referencial automática, joins flexibles.

11.2. Tipos y cuándo usar cada uno

Cada categoría de NoSQL tiene un modelo de datos distinto, optimizado para un patrón de acceso específico. Elegir una base de datos columnar para un caso de uso que

necesita documentos heterogéneos, o un grafo para un caso de uso que necesita caché, produce sistemas más complicados y lentos que un RDBMS bien diseñado.

- **Documental (MongoDB, CouchDB, Firestore):** cada registro es un documento JSON/BSON autocontenido, sin esquema rígido. Las relaciones se manejan embebiendo documentos dentro de otros (desnormalización intencional) o con referencias manuales. Ideal para: catálogos de productos, CMS, perfiles de usuario con atributos variables, datos que se leen y escriben como unidad. Limitación: joins entre colecciones son costosos y no garantizados ACID en todas las versiones.
- **Clave-valor (Redis, DynamoDB, Memcached):** el modelo más simple — un mapa de clave a valor opaco. Acceso $O(1)$ por clave exacta. Redis enriquece el modelo con tipos de valor (strings, listas, sets, sorted sets, hashes, streams). Ideal para: caché de sesiones, rankings en tiempo real, colas de trabajo, contadores atómicos, pub/sub. Limitación: no soporta consultas por valor, solo por clave exacta.
- **Columnar (Cassandra, HBase, ClickHouse):** los datos se almacenan agrupados por columna, no por fila. Optimizado para leer muchas filas de pocas columnas (analítica) y para escritura masiva distribuida. Cassandra particiona datos por una partition key y los ordena por clustering keys. Ideal para: series de tiempo, logs de eventos, métricas de IoT, historial de actividad. Limitación: el modelo de consulta es rígido — diseña las tablas en función de las queries, no al revés.
- **Grafos (Neo4j, Amazon Neptune, JanusGraph):** nodos (entidades) y aristas (relaciones) con propiedades en ambos. Traversar relaciones es $O(1)$ por arista, independiente del tamaño total del grafo. Ideal para: detección de fraude (encontrar patrones en redes de transacciones), recomendaciones (“quienes compraron X también compraron Y”), gestión de identidades y acceso. Limitación: no escala horizontalmente tan bien como otros NoSQL; menos herramientas y ecosistema.

11.3. Teorema CAP

El teorema CAP (Eric Brewer, 2000) establece una limitación fundamental de los sistemas distribuidos: ningún sistema puede garantizar simultáneamente las tres propiedades. Durante una partición de red (falla de comunicación entre nodos), el sistema debe elegir entre consistencia y disponibilidad.

- **C (Consistency — Consistencia):** todos los nodos del sistema ven exactamente los mismos datos al mismo tiempo. Una escritura en el nodo A es inmediatamente visible al leer en el nodo B.
- **A (Availability — Disponibilidad):** el sistema siempre responde a las requests, aunque la respuesta pueda ser con datos desactualizados. Nunca retorna error por indisponibilidad.
- **P (Partition Tolerance — Tolerancia a particiones):** el sistema sigue funcionando aunque haya fallas de red que dividan el cluster en grupos que no pueden comunicarse entre sí.

En la práctica, la Tolerancia a Particiones (P) es **no negociable** en cualquier sistema distribuido real: las redes fallan. Por lo tanto, la elección real es entre C y A durante una partición:

💡 CAP y bases de datos reales

- **CP** (Consistency + Partition Tolerance): MongoDB, HBase, ZooKeeper. Durante una partición, prefieren rechazar requests (perder disponibilidad) antes que servir datos inconsistentes. El sistema puede estar “temporalmente caído” pero los datos son siempre correctos.
- **AP** (Availability + Partition Tolerance): Cassandra, DynamoDB, CouchDB. Durante una partición, siguen respondiendo con los datos que tienen, aunque puedan estar desactualizados. Usan **consistencia eventual**: con el tiempo, todos los nodos convergen al mismo valor.
- **CA** (Consistency + Availability sin Partition Tolerance): solo posible en un único nodo. En la práctica, un RDBMS en un solo servidor. En cuanto hay replicación, P está siempre presente.

La consistencia eventual significa: “si no hay nuevas escrituras, eventualmente todos los nodos tendrán el mismo valor”. El tiempo de convergencia depende del sistema y la carga. Es aceptable cuando los usuarios pueden tolerar ver datos levemente desactualizados (timelines de redes sociales, contadores aproximados).

Listing 11.1: MongoDB — CRUD básico

```

1 // Insertar documento
2 db.productos.insertOne({
3     nombre: "Laptop",
4     precio: 15000,
5     specs: { ram: "16GB", ssd: "512GB" },
6     tags: ["electrónica", "computadoras"]
7 });
8
9 // Consultar con filtro anidado
10 db.productos.find(
11     { "specs.ram": "16GB", precio: { $lt: 20000 } },
12     { nombre: 1, precio: 1, _id: 0 }
13 );
14
15 // Agregar con $group (equivale a GROUP BY)
16 db.ventas.aggregate([
17     { $match: { fecha: { $gte: new Date("2024-01-01") } } },
18     { $group: { _id: "$region", total: { $sum: "$monto" } } },
19     { $sort: { total: -1 } }
20 ]);

```

Listing 11.2: Redis — caché y estructuras de datos

```

1 # String con TTL
2 SET session:user:123 '{"nombre":"Ana","rol":"admin"}' EX 3600
3
4 # Contador atómico

```

```
5 INCR visitas:articulo:456
6 EXPIRE visitas:articulo:456 86400
7
8 # Lista (cola de trabajo)
9 LPUSH cola:emails "job:uuid-1234"
10 RPOP cola:emails
11
12 # Set ordenado (ranking)
13 ZADD ranking 1500 "usuario:ana"
14 ZADD ranking 2300 "usuario:carlos"
15 ZREVRANGE ranking 0 9 WITHSCORES
```



Preguntas típicas

1. ¿Cuándo elegiría MongoDB sobre PostgreSQL para almacenar datos de productos?
2. Explica el teorema CAP y dónde encajan Redis y Cassandra.
3. ¿Cuál es la diferencia entre consistencia fuerte y eventual?

Capítulo 12

Diseño de Base de Datos — Caso Práctico

12.1. Proceso de diseño

El diseño de una base de datos es un proceso iterativo con fases bien definidas. Saltarse las etapas conceptuales e ir directamente al DDL produce esquemas que son difíciles de mantener, escalar y corregir. Un buen diseño se valida con casos de uso reales antes de escribir una sola línea de SQL.

Las seis fases del diseño

1. **Requisitos:** entrevistas con stakeholders — ¿qué datos necesitan almacenar? ¿qué consultas harán?
2. **Modelo conceptual:** diagrama ER/UML — entidades, atributos, relaciones (independiente del DBMS)
3. **Modelo lógico:** traducir ER a tablas, definir tipos de datos, PK/FK, constraints
4. **Normalización:** aplicar 3NF mínimo; eliminar redundancias y anomalías
5. **Modelo físico:** índices, particionamiento, tipos específicos del DBMS (JSONB, arrays)
6. **Revisión de rendimiento:** EXPLAIN ANALYZE, pruebas de carga con datos realistas

El modelo conceptual y lógico son agnósticos al motor; el modelo físico depende del DBMS elegido.

12.2. Caso: Sistema de Biblioteca

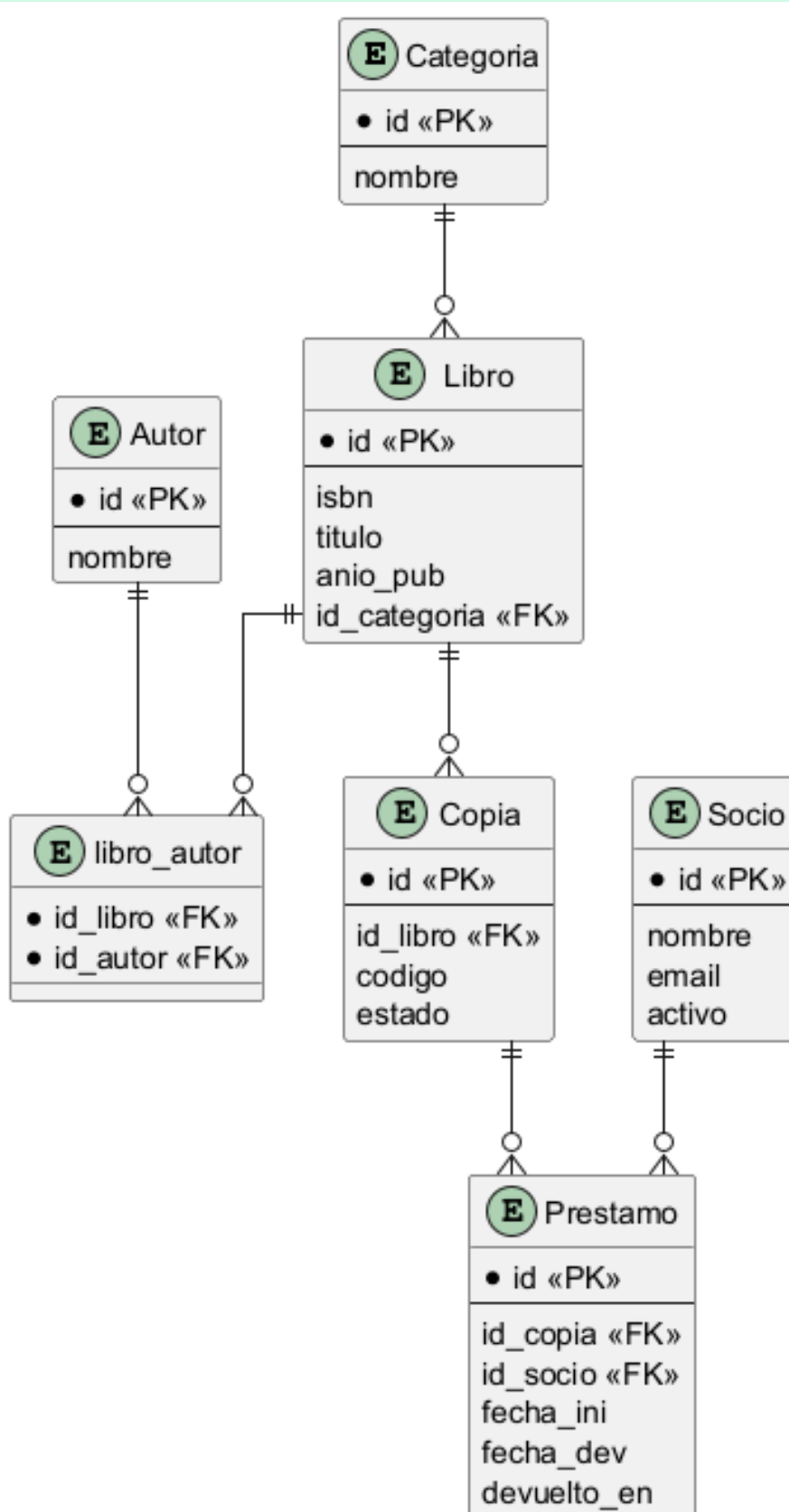
Ejemplo completo: desde requisitos hasta DDL optimizado.

Requisitos: registrar libros, copias físicas, socios, préstamos y devoluciones. Una copia puede prestarse y devolverse múltiples veces a lo largo del tiempo. Un libro puede tener varios autores; un autor puede haber escrito varios libros.

Entidades identificadas: Libro, Copia, Socio, Prestamo, Autor, Categoria.

Relaciones clave:

- Libro tiene muchas Copias físicas (1:N) — un libro puede existir en varios ejemplares
- Libro tiene muchos Autores (M:N via tabla intermedia `libro_autor`)
- Copia puede estar en muchos Préstamos a lo largo del tiempo (1:N)
- Socio tiene muchos Préstamos (1:N)

 ER — Sistema Biblioteca

Listing 12.1: DDL completo — Sistema Biblioteca

```

1 CREATE TABLE categorias (
2     id          SERIAL PRIMARY KEY,
3     nombre     VARCHAR(80) NOT NULL UNIQUE
4 );
5
6 CREATE TABLE libros (
7     id          SERIAL PRIMARY KEY,
8     isbn        VARCHAR(13) UNIQUE NOT NULL,
9     titulo      VARCHAR(200) NOT NULL,
10    anio_pub     SMALLINT,
11    id_categoria INTEGER REFERENCES categorias(id)
12 );
13
14 CREATE TABLE autores (
15     id          SERIAL PRIMARY KEY,
16     nombre     VARCHAR(150) NOT NULL
17 );
18
19 CREATE TABLE libro_autor (
20     id_libro    INTEGER REFERENCES libros(id) ON DELETE CASCADE,
21     id_autor    INTEGER REFERENCES autores(id) ON DELETE CASCADE,
22     PRIMARY KEY (id_libro, id_autor)
23 );
24
25 CREATE TABLE copias (
26     id          SERIAL PRIMARY KEY,
27     id_libro    INTEGER NOT NULL REFERENCES libros(id),
28     codigo      VARCHAR(20) UNIQUE NOT NULL,
29     estado      VARCHAR(20) DEFAULT 'disponible'
30     CHECK (estado IN ('disponible', 'prestada', 'dannada', '
31                     baja'))
32 );
33
34 CREATE TABLE socios (
35     id          SERIAL PRIMARY KEY,
36     nombre     VARCHAR(150) NOT NULL,
37     email      VARCHAR(200) UNIQUE NOT NULL,
38     activo     BOOLEAN DEFAULT TRUE,
39     creado_en  DATE DEFAULT CURRENT_DATE
40 );
41
42 CREATE TABLE prestamos (
43     id          SERIAL PRIMARY KEY,
44     id_copia    INTEGER NOT NULL REFERENCES copias(id),
45     id_socio    INTEGER NOT NULL REFERENCES socios(id),
46     fecha_ini   DATE NOT NULL DEFAULT CURRENT_DATE,
47     fecha_dev   DATE NOT NULL,
48     CONSTRAINT chk_fechas CHECK (fecha_dev > fecha_ini)
49 );

```


Listing 12.2: Consultas operacionales sobre el diseño

```
1  -- Copias disponibles por categoría
2  SELECT c.nombre AS categoria, COUNT(cp.id) AS copias_disponibles
3  FROM   categorias c
4  JOIN   libros l   ON l.id_categoria = c.id
5  JOIN   copias cp  ON cp.id_libro = l.id AND cp.estado = 'disponible'
6  GROUP BY c.nombre
7  ORDER BY copias_disponibles DESC;
8
9  -- Socios con préstamos vencidos
10 SELECT s.nombre, s.email, p.fecha_dev, l.titulo
11 FROM   prestamos p
12 JOIN   socios s   ON p.id_socio = s.id
13 JOIN   copias cp  ON p.id_copia = cp.id
14 JOIN   libros l   ON cp.id_libro = l.id
15 WHERE  p.devuelto_en IS NULL
16        AND p.fecha_dev < CURRENT_DATE
17 ORDER BY p.fecha_dev;
```



Preguntas típicas

1. ¿Cómo modelarías la diferencia entre un libro (título) y una copia física en una biblioteca?
2. ¿Qué índices agregarías a este diseño para optimizar las consultas más frecuentes?
3. ¿Cómo manejarías el historial de préstamos si un socio puede tomar el mismo libro varias veces?

Capítulo 13

Administración de Bases de Datos

13.1. Backups

El backup es la operación más importante de la administración de bases de datos y, paradójicamente, la más frecuentemente ignorada hasta que ocurre un desastre. La regla más importante: **un backup no probado no existe**. Un dump corrupto, incompleto o que no puede restaurarse es exactamente igual a no tener backup.

PostgreSQL ofrece dos niveles de backup con propósitos distintos:

- **Backup lógico:** extrae los datos como SQL o en formato binario propio de PostgreSQL. Portable entre versiones y sistemas operativos. Puede restaurar tablas o bases individuales. Herramientas: `pg_dump` (una BD) y `pg_dumpall` (todo el cluster + roles globales).
- **Backup físico:** copia los archivos binarios del cluster de PostgreSQL tal como están en disco. Más rápido para BDs grandes. Requiere la misma versión de PostgreSQL al restaurar. Herramienta: `pg_basebackup`. Necesario para Point-In-Time Recovery (PITR).

Estrategia de backup recomendada

Una estrategia robusta combina ambos tipos:

- **pg_basebackup semanal:** copia física completa del cluster. Base para PITR.
- **WAL Archiving continuo:** archiva cada segmento de WAL (16MB cada uno) a un almacenamiento externo. Permite restaurar a cualquier segundo entre el último basebackup y ahora.
- **pg_dump diario por BD crítica:** backup lógico para restauración selectiva de tablas, más fácil de hacer en ambientes de desarrollo/staging.

Prueba de restauración mensual: restaurar un backup en un servidor de prueba y verificar que los datos son correctos y la aplicación funciona. Sin esta prueba, no sabes si el backup funciona hasta que lo necesitas en producción.

Errores comunes en backups

- Backups en el mismo disco que los datos: si el disco falla, se pierde todo.
- No incluir `pg_dumpall` en la estrategia: los roles y tablespaces globales solo

existen ahí.

- Usar **VACUUM FULL** en producción sin planificación: bloquea la tabla completamente mientras reescribe; puede durar horas en tablas grandes.
- Confiar en **pg_dump** solo: sin WAL archiving, el RPO (punto de recuperación) es el momento del último dump, que puede ser hace 24 horas.

13.2. Usuarios, roles y permisos

PostgreSQL usa **roles** como unidad unificada de identidad y permisos. Un rol con el atributo **LOGIN** es lo que coloquialmente llamamos “usuario”; sin ese atributo, es un grupo de permisos que se asigna a otros roles. Esta unificación simplifica la administración: en lugar de gestionar usuarios y grupos por separado, todo es un rol.

El modelo de permisos en PostgreSQL es **jerárquico y aditivo**: un rol hereda los permisos de todos los roles que se le asignan. Esto permite construir una jerarquía: rol **readonly** → se asigna a **reportes_user**; si además se necesita escritura, se asigna **readwrite** al mismo usuario sin quitar nada.

El **principio de mínimo privilegio** dicta que cada rol tenga exactamente los permisos que necesita para su función, ni más. Una aplicación web que solo lee datos no debe tener permiso de **DROP TABLE**. Un usuario de reportes no debe poder **INSERT**. Esto limita el daño en caso de compromiso de credenciales o bug en la aplicación.

Los permisos se otorgan a distintos niveles de granularidad:

- **Conexión a BD:** **GRANT CONNECT ON DATABASE**
- **Uso de schema:** **GRANT USAGE ON SCHEMA**
- **Tablas:** **GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE**
- **Columnas:** **GRANT SELECT (col1, col2) ON TABLE** — oculta columnas sensibles
- **Filas:** Row Level Security (RLS) — cada fila puede tener su propia política de acceso

13.3. Monitoreo básico

El monitoreo de una base de datos tiene dos objetivos: detectar problemas *antes* de que impacten al usuario (monitoreo proactivo) y diagnosticar problemas cuando ya ocurrieron (monitoreo reactivo). Las vistas del sistema de PostgreSQL ofrecen visibilidad en tiempo real sin necesidad de herramientas externas.

Las señales de alerta más importantes a monitorear continuamente:

- **Conexiones activas:** si se acerca al límite de **max_connections**, nuevas conexiones son rechazadas. Considera un connection pooler (PgBouncer).
- **Transacciones largas:** una transacción abierta por horas bloquea el **VACUUM** y acumula bloat. Detectar y terminar las idle in transaction es crítico.

- **Dead tuples:** filas muertas acumuladas por MVCC. Si autovacuum no las limpia a tiempo, la tabla crece sin control y las queries se vuelven lentas.
- **Queries lentas:** con `pg_stat_statements`, identificar las queries con mayor tiempo total de ejecución y optimizarlas.
- **Locks:** un lock no otorgado (waiting) indica contención. Si se acumula una cadena de locks, puede paralizar la aplicación.

Listing 13.1: Backups con `pg_dump` y restauración

```

1 -- Backup lógico (formato custom, comprimido)
2 pg_dump -U postgres -Fc -f backup_mibd_20260516.dump mibd
3
4 -- Backup lógico en SQL plano
5 pg_dump -U postgres -f backup_mibd.sql mibd
6
7 -- Restaurar desde formato custom
8 pg_restore -U postgres -d mibd_nueva backup_mibd_20260516.dump
9
10 -- Restaurar SQL plano
11 psql -U postgres -d mibd_nueva -f backup_mibd.sql
12
13 -- Backup de todas las bases + roles globales
14 pg_dumpall -U postgres -f cluster_completo.sql

```

Listing 13.2: Monitoreo — conexiones, locks y queries lentas

```

1 -- Conexiones activas
2 SELECT pid, username, state, query_start,
3        LEFT(query, 100) AS query_corta
4 FROM   pg_stat_activity
5 WHERE  state != 'idle'
6 ORDER BY query_start;
7
8 -- Terminar conexión colgada
9 SELECT pg_terminate_backend(pid)
10 FROM   pg_stat_activity
11 WHERE  state = 'idle in transaction'
12 AND    query_start < NOW() - INTERVAL '10 minutes';
13
14 -- Ver locks activos
15 SELECT pid, relation::regclass, mode, granted
16 FROM   pg_locks
17 WHERE  NOT granted;

```

Listing 13.3: `pg_stat_statements` — queries más lentas

```

1 -- Activar extensión (requiere superuser, reinicio de PG)
2 CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
3
4 -- Top 10 queries más lentas por tiempo promedio
5 SELECT LEFT(query, 120) AS query,
6        calls,

```

```
7      ROUND((total_exec_time / calls)::numeric, 2) AS avg_ms,  
8      ROUND(total_exec_time::numeric, 2) AS total_ms  
9 FROM   pg_stat_statements  
10 ORDER BY avg_ms DESC  
11 LIMIT  10;  
12  
13 -- Resetear estadísticas  
14 SELECT pg_stat_statements_reset();
```



Preguntas típicas

1. ¿Cuál es la diferencia entre `pg_dump` y `pg_basebackup`?
2. ¿Cómo identificarías una query lenta en producción sin bajar el servidor?
3. ¿Qué es el principio de mínimo privilegio aplicado a roles de base de datos?

Capítulo 14

Preguntas de Entrevista DBA

Este capítulo consolida las preguntas de todos los capítulos más preguntas adicionales, organizadas por tema. Las respuestas completas están en los capítulos correspondientes; este capítulo sirve para repasar y autoevaluarse.

14.1. SQL

Preguntas típicas

1. ¿Cuál es el orden lógico de evaluación de una query SELECT?
2. ¿Qué diferencia hay entre UNION y UNION ALL?
3. ¿Cómo funciona un SELF JOIN? Da un ejemplo.
4. ¿Qué es una subquery correlacionada? ¿Cuándo es problemática?
5. ¿Cuándo usarías EXISTS en lugar de IN?
6. ¿Qué devuelve COUNT(*) vs COUNT(columna)?
7. ¿Cuál es la diferencia entre TRUNCATE, DELETE y DROP?
8. Escribe una query que encuentre el segundo salario más alto.
9. ¿Cómo eliminas duplicados de una tabla manteniendo una sola copia?
10. ¿Qué es una vista materializada? ¿Cuándo la usarías?

14.2. Índices y rendimiento

Preguntas típicas

1. ¿Qué es un índice cubriente (covering index)?
2. ¿Por qué LIKE '%texto' no usa el índice B-tree pero LIKE 'texto%' sí?
3. ¿Cuándo NO deberías crear un índice?
4. ¿Qué es un index scan vs bitmap index scan?
5. ¿Cómo afectan los índices a las operaciones de escritura masiva?

14.3. Transacciones y concurrencia

Preguntas típicas

1. ¿Qué es un phantom read? ¿Qué nivel de aislamiento lo previene?
2. ¿Cuál es la diferencia entre optimistic y pessimistic locking?
3. ¿Cómo funciona MVCC en PostgreSQL?
4. ¿Qué es un dirty read?
5. Explica el problema de non-repeatable read.
6. ¿Cómo PostgreSQL detecta y resuelve deadlocks?

14.4. Modelado y diseño

Preguntas típicas

1. ¿Cuándo desnormalizarías intencionalmente una tabla?
2. ¿Cuál es la diferencia entre 3NF y BCNF?
3. ¿Cómo modelarías un sistema de permisos basado en roles (RBAC)?
4. ¿Cuándo usarías un UUID como PK en lugar de un entero?
5. ¿Cómo diseñarías el schema para un sistema de auditoría (audit log)?

14.5. NoSQL y arquitectura distribuida

Preguntas típicas

1. ¿Cuándo elegiría Cassandra sobre PostgreSQL?
2. ¿Qué es sharding y cuándo se necesita?
3. ¿Cuál es la diferencia entre replicación sincrónica y asincrónica?
4. ¿Qué significa consistencia eventual? ¿En qué situaciones es aceptable?
5. ¿Cómo implementarías caché con Redis para reducir carga en PostgreSQL?

14.6. Administración y operaciones

Preguntas típicas

1. ¿Qué harías si la BD está lenta en producción a las 3am?
2. ¿Cómo harías una migración de schema sin downtime?
3. ¿Qué es WAL (Write-Ahead Log) y por qué existe?
4. ¿Cómo harías rollback de una migración ya aplicada?

5. ¿Qué estrategia de backup usarías para una BD de 500GB?